

A typed API over a tenant-safe database

The win: open any file of epic 1 and know why it is shaped that way, including the twelve bugs the type system could not see.

How to use this page. Every code block is cut from the repo at commit d4eea6f, so what you read is what runs. Read a section, then open the named file and find the code in its real surroundings.

```
./init.sh                # boots Postgres + the api, generates local secrets
make check               # the gate: lint, types, unit, integration, dead
code, duplication, e2e
sh drills/lesson-1/check.sh # your drills: RED until you solve them

Try it: curl -s -X POST localhost:3000/orgs -H 'x-dev-user: you@example.com' -H
'content-type: application/json' -d '{"name": "Mine"}'
```

The map: one request, five files

- 1 · One schema, two jobs
- 2 · Privacy without `private` : the closure
- 3 · Parse at the boundary, in both directions
- 4 · Let the database do what it is good at
- 5 · The HTTP layer: a handler cannot forget to validate
- 6 · Identity: a function type, and failing closed
- 7 · A session token, and a function that returns its failure
- 8 · Configuration, startup and shutdown
- 9 · Least privilege in the database
- 10 · What the type system could not see

Your turn: three drills

Recall, then test yourself

The map: one request, five files

Epic 1 built the part of the system that is ordinary on purpose: a typed HTTP API over Postgres. Follow one request, `POST /orgs/:orgId/workspaces`, and you have met every file worth knowing.

STEP	WHERE	WHAT HAPPENS
1	<code>apps/api/src/app.ts</code>	identity middleware: who is calling? No caller, no entry (401)
2	<code>apps/api/src/app.ts</code>	org middleware: is this caller a member of this org? If not, 404
3	<code>packages/contracts</code>	the body is parsed against a Zod schema; a bad field is a 400 that names it
4	<code>packages/db</code>	<code>db.forOrg(orgId).createWorkspace()</code> : the only door to tenant data
5	Postgres	constraints and foreign keys have the last word

Three packages, one direction of dependency: `contracts` knows nothing; `db` and `session-token` import it; `api` imports all three. Nothing imports `api`.

TYPESCRIPT (THIS REPO)	JAVA / SPRING	PYTHON / FASTAPI
<code>packages/*</code> with <code>exports</code> in <code>package.json</code>	Maven modules	packages in a uv/poetry workspace
Hono app built by a function, <code>buildApp({ db, identify, sessions })</code>	Spring context + <code>@RestController</code>	<code>FastAPI()</code> + <code>Depends</code>
dependencies are function ARGUMENTS	dependency injection container	<code>Depends(...)</code>

The biggest style difference from Spring: there is no container, no annotations, no reflection. A dependency is a parameter. A test passes a different argument. That is the whole dependency-injection story in this codebase.

1 • One schema, two jobs

Lesson 0 ended on *types vanish at runtime*. This is the answer. A Zod schema is a runtime value that can check data; `z.infer` derives the static type from it. One definition, so the check and the type cannot drift apart.

FROM `packages/contracts/src/index.ts`

```
export const Id = z.uuid();
/**
 * Trimmed, 1-200 characters, no control characters. Used on the way IN (before a write)
and on the
 * way out. Control characters are refused here because Postgres text cannot hold a NUL
byte: without
 * this, a body that satisfies the contract would still fail in the database as a 500.
 */
export const Name = z
  .string()
  .trim()
  .min(1)
  .max(200)
  .regex(/^\P{Cc}*/u, "must not contain control characters");
const Timestamp = z.iso.datetime();

export const User = z.object({ id: Id, email: z.email().max(320), name: Name });
export type User = z.infer<typeof User>;

export const Org = z.object({ id: Id, name: Name, createdAt: Timestamp });
export type Org = z.infer<typeof Org>;

export const Workspace = z.object({ id: Id, orgId: Id, name: Name, createdAt: Timestamp
});
export type Workspace = z.infer<typeof Workspace>;
```

Three TypeScript things are happening in that excerpt:

- **A value and a type share a name.** `export const Org` and `export type Org` do not clash: TypeScript keeps values and types in two separate namespaces. `Org.parse(x)` uses the value; `const o: Org` uses the type. Java has no equivalent; it is the idiom for "schema + its type".
- **`typeof Org`** here is the *type-level* `typeof`: "the static type of this value". (The runtime `typeof` from Lesson 0 only knows eight JavaScript types. Same keyword, two languages.)
- **`z.strictObject`** (used for request bodies, below) rejects unknown keys, so a typo like `nmae` is an error, not silently ignored.

FROM `packages/contracts/src/index.ts`

```
export const CreateOrgBody = z.strictObject({ name: Name });
export const CreateWorkspaceBody = z.strictObject({ name: Name });
export const CreateDocumentBody = z.strictObject({ title: Name });
/** PUT /orgs/:orgId/members: makes the user with this email a member at `role`, or
changes their role. Owners only. */
export const SetMemberBody = z.strictObject({ email: User.shape.email, role: Role });
/**
 * PUT /documents/:id/shares (F25): shares ONE document with the user with this email, or
changes their share. Never
 * `owner`: a share lets someone work on a document, never run the org. Owners of the
document's org only.
```

```

*/
export const ShareBody = z.strictObject({ email: User.shape.email, role:
z.enum(["editor", "viewer"]) });

```

Typescript (this repo)

Java / Spring

Python / FastAPI

Zod schema + `z.infer`

DTO class + Bean Validation annotations

pydantic `BaseModel`

explicit `.parse()` at each boundary

framework binds and validates for you

framework binds and validates for you

types erased; only the schema exists at runtime

class exists at runtime

class exists at runtime

CASE STUDY · FOUND BY REVIEW

A name that satisfied the contract still crashed the database.

The first `Name` was `z.string().trim().min(1).max(200)`. Two reviewers independently sent a name containing a NUL byte. The contract said yes, Postgres said no (text cannot hold NUL), and the client got a 500. The fix is the `.regex(/^\\P{Cc}*$/u)` line: `\\P{Cc}` means "not a control character", and the `u` flag turns on Unicode property escapes. *A contract must be at least as strict as the strictest system behind it.*

2 · Privacy without private: the closure

The most important rule in a multi-tenant system: **no query may forget the org filter**. In Java you would reach for a `private` field and code review. Here the rule is made *unwritable*.

FROM `packages/db/src/index.ts`

```

export type Db = {
  migrate(): Promise<void>;
  appliedMigrations(): Promise<string[]>;
  /** Finds the user with this email or creates one. Emails are compared case-
  insensitively. */
  upsertUser(input: { email: string; name: string }): Promise<User>;
  /** E8.1: a new user with a password, in one statement. "taken": the email belongs to
  someone (with or without a password). */
  signUp(input: { email: string; name: string; passwordHash: string }): Promise<User |
  "taken">;
  /** The user with this email and their password hash; undefined when there is none, or
  they have no password. */
  credentialsFor(email: string): Promise<{ user: User; passwordHash: string } |
  undefined>;
  /**

```

```

    * Records a signed-in browser by the SHA-256 of its token, and forgets this user's
sessions that have expired. The
    * same statement writes the sign-in into the audit trail of every org the user is a
member of (F26).
    */
    startSession(input: { userId: string; tokenHash: Buffer; ttlSeconds: number }):
Promise<void>;
    /** Who holds this session, if it exists and has not expired. A pure READ: asked on
every request. */
    userForSession(tokenHash: Buffer): Promise<User | undefined>;
    /** Signs a browser out: the row goes, so the token stops working on the very next
request. */
    endSession(tokenHash: Buffer): Promise<void>;
    /** Creates the org and makes `ownerId` its owner, atomically: an org never exists
without an owner. */
    createOrg(input: { name: string; ownerId: string }): Promise<Org>;
    listOrgsFor(userId: string, page?: PageInput): Promise<Page<Org> | undefined>;
    /** The org and this user's role in it, but only if they are a member. "Not a member"
and "no such org" look the same. */
    getOrgForMember(orgId: string, userId: string): Promise<{ org: Org; role: Role } |
undefined>;
    /**
    * The document and this user's role on it, but only if they are a member of its org or
it is shared with them
    * (E8.3: the higher of the two). Used where the path names no org.
    */
    getDocumentForMember(documentId: string, userId: string): Promise<{ document: Document;
role: Role } | undefined>;
    /**
    * E8.2 (F24): this user's role on the document, if it is `orgId`'s and they are a
member of that org or it is shared
    * with them (E8.3); undefined otherwise. The sync server asks it when a peer joins and
when an access change is announced.
    */
    roleIn(orgId: string, documentId: string, userId: string): Promise<Role | undefined>;
    /** Resolves if the database answers a query, rejects otherwise. */
    ping(): Promise<void>;
    /**
    * E9.5 (F31): one hit on `key` under `rule`, counted in Postgres so every api instance
shares the count (limit.ts).
    * Rejects when the database is away: the caller decides whether that fails open or
closed. E9.6's seam.
    */
    take(key: string, rule: Rule): Promise<Verdict>;
    /** Loading and saving a document's tree, for the sync server. Every call names the
org. */
    documentStore(): DocumentStore;
    /** Claiming and finishing jobs, for the worker. Every call names the org. */
    jobStore(): JobStore;
    /** The git peer's inbox (E5.3a): the stack's one repo, so no org. */
    gitStore(): GitStore;
    /** The ONLY way to reach tenant data: every query it runs is filtered by this org. */
    forOrg(orgId: string): OrgScope;

```

```
close(): Promise<void>;
};
```

FROM `packages/db/src/index.ts`

```
export function createDb({ connectionString, schema }: { connectionString: string;
schema?: string }): Db {
  // `schema` goes into libpq's space-separated startup options, where a space would
  smuggle in
  // extra `-c` settings (the review set session_replication_role this way). Plain
  identifiers only.
  if (schema !== undefined && !IDENTIFIER.test(schema)) {
    throw new Error(`invalid schema name: ${JSON.stringify(schema)}`);
  }

  // The pool lives in this closure and is never returned: there is no way to run an
  unscoped query from outside.
  const pool = new Pool({
    connectionString,
    application_name: "noon-db",
    ...(schema === undefined ? {} : { options: `-c search_path=${schema}` }),
  });
  // Postgres restarting, or killing an idle connection, makes the pool emit 'error'. An
  'error'
  // event with no listener is an uncaught exception in Node: one dropped connection
  would take
  // the whole process down. The pool has already discarded the client; only the message
  is logged
  // (the full error object carries the connection password).
  pool.on("error", (err) => {
    process.stderr.write(`${JSON.stringify({ level: "warn", source: "db", message: `idle
connection lost: ${err.message}` })}\n`);
  });

  /** `via`: a client inside a transaction; the pool otherwise. */
```

`pool` is a local variable of `createDb`. The functions in the returned object *close over* it: they can use it, nothing else can reach it. There is no field to make public by mistake, no reflection to bypass it, and no subclass. That is a **closure**, and it is how this codebase does encapsulation.

TYPESCRIPT (THIS REPO)	JAVA / SPRING	PYTHON / FASTAPI
function returning an object of functions; state in the closure	class with <code>private</code> <code>final</code> fields	class with <code>_underscore</code> convention
<code>type Db = { ... }</code> describes the shape	<code>interface Db</code>	<code>Protocol</code>
truly unreachable	reachable by reflection	reachable by anyone

Why not a class? TypeScript's `private` is a compile-time note that vanishes at runtime (JavaScript's `#field` is real, but then every method needs `this`, and `this` gets lost the moment you pass a method as a callback; you will meet that lint rule in section 8). An object of closures has no `this` at all.

And a test proves the door stays shut. `@ts-expect-error` means "the next line MUST be a type error"; if it ever compiles, the typecheck layer fails:

FROM `packages/db/src/scope.typecheck.test.ts`

```
function unscopedAccessIsImpossible(db: Db): void {
  // @ts-expect-error the connection pool is not reachable from outside the package
  void db.pool;
  // @ts-expect-error there is no raw query method
  void db.query("select * from workspaces");
  // @ts-expect-error tenant data cannot be listed without naming an org
  void db.listWorkspaces();
  // @ts-expect-error a scope needs an org id
  void db.forOrg();
}
```

Three layers, not one. The closure makes the unscoped query unwritable; the compile-time test notices if someone adds a door; and the database refuses a document whose org differs from its workspace's org (a composite foreign key, section 4). Each layer was mutation-checked: remove it, watch a test go red, put it back.

3 · Parse at the boundary, in both directions

The pg driver returns rows typed as `any`. Rather than trust it, every row is parsed once, where it enters our code, and converted from the database's `snake_case` to the contract's shape:

FROM `packages/db/src/index.ts`

```
// Rows arrive as `any` from the driver. Each is parsed once, here, at the database
boundary.
const timestamp = z.date().transform((d) => d.toISOString());
const UserRow = z.object({ id: z.string(), email: z.string(), name: z.string()
}).transform((r): User => User.parse(r));
const OrgRow = z.object({ id: z.string(), name: z.string(), created_at: timestamp })
  .transform((r): Org => Org.parse({ id: r.id, name: r.name, createdAt: r.created_at }));
const WorkspaceRow = z.object({ id: z.string(), org_id: z.string(), name: z.string(),
created_at: timestamp })
  .transform((r): Workspace => Workspace.parse({ id: r.id, orgId: r.org_id, name: r.name,
createdAt: r.created_at }));
const DocumentRow = z.object({ id: z.string(), org_id: z.string(), workspace_id:
z.string(), title: z.string(), created_at: timestamp }) // content and seq are read only
by documentStore()
  .transform((r): Document =>
```

```
Document.parse({ id: r.id, orgId: r.org_id, workspaceId: r.workspace_id, title:
r.title, createdAt: r.created_at }));
```

`.transform()` makes a schema whose *input* type and *output* type differ: in goes a row with a `Date`, out comes an `Org` with an ISO string. The annotation `(r): Org =>` makes the compiler check that the transform really produces the contract.

CASE STUDY · FOUND BY REVIEW

Validate BEFORE the write, not only after the read.

The first version parsed rows only on the way out. The database check was `length(name) >= 1`; the contract trims first. So `" "` was *stored* (length 3) and then *unreadable* (trims to empty). One such row made every later `listWorkspaces()` for that org throw, forever, and nothing in the API could delete it. Two reviewers found it independently.

The fix has two halves: inputs go through the same `Name.parse()` **before** the insert, and the column check became `name = btrim(name)`. The rule: *a row the reader cannot parse must never be storable*.

FROM `packages/db/src/index.ts`

```
// Inputs are parsed with the SAME contract the reader uses, BEFORE the write: a row
that
// cannot be read back must never be stored (it would make every later list throw).
createOrg: async ({ name, ownerId }) => {
  if (!isId(ownerId)) throw new Error("cannot create an org: invalid owner id");
  return exactlyOne(
    OrgRow,
    // One statement, so it is atomic without an explicit transaction.
    "with o as (insert into orgs (name) values ($1) returning *), " +
    "m as (insert into memberships (org_id, user_id, role) select o.id, $2, 'owner'
from o) select * from o",
    [Name.parse(name), ownerId],
  );
},
```

4 · Let the database do what it is good at

Application code checks first; the database has the last word. Three pieces of SQL carry real weight.

A composite foreign key makes "a document's org equals its workspace's org" a fact of the schema, not a hope about the code:

FROM packages/db/migrations/0001_init.sql

```
create table documents (  
  id          uuid primary key default gen_random_uuid(),  
  org_id      uuid not null references orgs (id) on delete cascade,  
  workspace_id uuid not null,  
  title       text not null check (title = btrim(title) and length(title) between 1 and  
200),  
  created_at  timestampz not null default now(),  
  -- The database, not the application, guarantees a document's org matches its  
workspace's org.  
  foreign key (workspace_id, org_id) references workspaces (id, org_id) on delete cascade  
);
```

insert ... select creates the row only if the workspace exists *in this org*. No check-then-insert, so no race:

FROM packages/db/src/index.ts

```
createDocument: async ({ workspaceId, title }) => {  
  const cleanTitle = Name.parse(title);  
  if (!orgExists || !isId(workspaceId)) return undefined;  
  return one(  
    DocumentRow,  
    // insert ... select: the row is only created if the workspace exists in this  
org.  
    "insert into documents (org_id, workspace_id, title) " +  
      "select w.org_id, w.id, $3 from workspaces w where w.org_id = $1 and w.id =  
$2 returning *",  
    [orgId, workspaceId, cleanTitle],  
  );  
},
```

One statement, two writes. An org must never exist without an owner. A data-modifying CTE does both inserts atomically, with no explicit transaction (Postgres runs every data-modifying **with** exactly once, to completion, whether or not its result is read):

FROM packages/db/src/index.ts

```
createOrg: async ({ name, ownerId }) => {  
  if (!isId(ownerId)) throw new Error("cannot create an org: invalid owner id");  
  return exactlyOne(  
    OrgRow,  
    // One statement, so it is atomic without an explicit transaction.  
    "with o as (insert into orgs (name) values ($1) returning *), " +  
      "m as (insert into memberships (org_id, user_id, role) select o.id, $2, 'owner'  
from o) select * from o",  
    [Name.parse(name), ownerId],  
  );  
},
```

Keyset paging. `OFFSET` gets slower with every page and skips rows when data changes underneath it. A keyset cursor says "everything after this row":

FROM `packages/db/src/index.ts`

```
// Keyset paging on (created_at, id). The cursor carries Postgres's own text form of the
timestamp,
// because a JS Date keeps milliseconds while timestampz keeps microseconds: rounding
through a Date
// would skip or repeat rows created in the same millisecond.
const CURSOR_TS = /^\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2}(\\.\\d{1,6})?[+-]\\d{2}(:\\d{2})?$/;
const encodeCursor = (ts: string, id: string): string =>
Buffer.from(`${ts}|${id}`).toString("base64url");
function decodeCursor(cursor: string): { ts: string; id: string } | undefined {
  const [ts, id, ...rest] = Buffer.from(cursor, "base64url").toString("utf8").split("|");
  if (ts === undefined || id === undefined || rest.length > 0) return undefined;
  // Postgres prints the offset as "+00"; ISO 8601 (what Date.parse reads) needs
  "+00:00".
  const iso = ts.replace(" ", "T").replace(/([+-]\\d{2})$/, "$1:00");
  if (!CURSOR_TS.test(ts) || Number.isNaN(Date.parse(iso)) || !isId(id)) return
undefined;
  return { ts, id };
}
```

CASE STUDY · FOUND BY REVIEW

A JavaScript `Date` is too coarse to be a cursor.

Postgres `timestampz` keeps microseconds; a JS `Date` keeps milliseconds. Rows inserted in the same millisecond would be skipped or repeated if the cursor went through a `Date`. So the cursor carries *Postgres's own text* for the timestamp. Drill 2 is this bug, planted in a copy of the code.

TYPESCRIPT (THIS REPO)	JAVA / SPRING	PYTHON / FASTAPI
hand-written SQL, parameters as <code>\$1</code>	JPA / jOOQ / JdbcTemplate	SQLAlchemy
ordered <code>.sql</code> files + an advisory lock	Flyway / Liquibase	Alembic
rows parsed by Zod	entity mapping	ORM models

Why no ORM? A dozen queries do not pay for one, and the hard statements later in this project (the fenced journal append in epic 7) need exact SQL anyway. The trade: you read real SQL, and nothing generates it behind your back.

5 · The HTTP layer: a handler cannot forget to validate

FROM `apps/api/src/app.ts`

```
async function body<S extends z.ZodType>(c: Context, schema: S): Promise<z.infer<S>> {
  // Browsers may send text/plain cross-site WITHOUT a preflight. Demanding JSON closes
  that door
  // before any cookie-based auth exists to be abused through it.
  if (!/^application\/json\b/i.test(c.req.header("content-type") ?? "")) {
    throw new HTTPException(415, { res: fail(c, 415, "unsupported_media_type") });
  }
  let raw: unknown;
  try {
    raw = await c.req.json();
  } catch {
    throw new HTTPException(400, { res: fail(c, 400, "invalid_json") });
  }
  const parsed = schema.safeParse(raw);
  if (parsed.success) return parsed.data;
  throw new HTTPException(400, { res: fail(c, 400, "invalid_body", issuesOf(parsed.error,
    "body")) });
}

/** F27: the request's `Idempotency-Key`, if it sent one. A malformed one is refused,
  never ignored: the retry it was meant to make safe would not be. */
function idempotencyKey(c: Context): string | undefined {
  const key = c.req.header("idempotency-key");
  if (key === undefined) return undefined;
  if (IdempotencyKey.safeParse(key).success) return key;
  throw new HTTPException(400, { res: fail(c, 400, "invalid_body", [{ field:
    "idempotency-key", message: "1 to 255 printable ASCII characters" }]) });
}
```

Read the signature slowly: `<S extends z.ZodType>(c, schema: S): Promise<z.infer<S>>`. It is **generic over the schema**: pass `CreateOrgBody` and the return type is *that schema's* inferred type. So in a handler, `const { name } = await body(c, CreateOrgBody)` gives a `name: string` with no cast and no annotation. On failure it *throws* a ready-made 400, which keeps every handler's happy path a straight line:

FROM `apps/api/src/app.ts`

```
app.post("/orgs", async (c) => {
  const { name } = await body(c, CreateOrgBody);
  return c.json(await db.createOrg({ name, ownerId: c.var.user.id }), 201);
});
```

satisfies appears all over `app.ts`: `{ error: "not_found" } satisfies ErrorBody` asks the compiler to check the literal against the type *without changing the literal's own type*. A plain annotation would widen it; `as` would not check it at all.

Errors. Anything unexpected becomes a bare 500. Database errors echo input, table and constraint names, so none of it goes to the client; the log is one JSON object per line, so a newline inside a message cannot forge a second log entry:

FROM apps/api/src/app.ts

```
app.onError((err, c) => {
  if (err instanceof HTTPException) return err.getResponse();
  // Database errors echo input, table and constraint names: the client gets none of
  it. The log is
  // one JSON object per line, so a newline inside a message cannot forge a second log
  entry.
  process.stderr.write(`${JSON.stringify({ level: "error", method: c.req.method, path:
  c.req.path, message: err.message })}\n`);
  return fail(c, 500, "internal");
});
```

CASE STUDY • FOUND BY REVIEW

One request took the API from 58 MB to 806 MB.

Nothing limited the size of a request body. A reviewer sent one 200 MB POST and watched the container's memory. The fix is one middleware (`bodyLimit` , 64 KB, 413), plus a demand for `application/json` : a browser may send `text/plain` cross-site *without a preflight*, which becomes a cross-site write the day cookies exist.

6 • Identity: a function type, and failing closed

Who is calling? In development a header answers; epic 8 added sign-in as a second strategy of the same type. The interesting part is the *shape*: identity is a function type, and the app receives one as an argument.

FROM apps/api/src/identity.ts

```
/** Works out who is calling. Undefined means "nobody we recognise", which the app turns
into 401. */
export type Identify = (c: Context, db: Db) => Promise<User | undefined>;

/**
 * Development only (SPEC §2.16): the caller names themselves in a header and is created
on first
 * sight. It exists so that tenancy and attribution are real from epic 1. Since E8.1
every other
 * NODE_ENV knows only sign-in sessions; development keeps the header (scripts, the
tunnel demo, two
 * browser windows as two people) behind a real session.
 */
```

```

    * Known costs, accepted because this never runs outside a developer's machine: anyone
    who can reach
    * the api can be anyone and can create users, and every request WRITES (the upsert), so
    it takes a
    * row lock and cannot run against a read replica. `sessionIdentity` below is a pure
    read.
    */
export const devHeaderIdentity: Identify = async (c, db) => {
    const email = User.shape.email.safeParse(c.req.header("x-dev-user"));
    if (!email.success) return undefined;
    // The display name is the local part, cut to what `Name` allows: a valid address may
    have a
    // local part far longer than a name, and that must not turn a request into a 500.
    const local = (email.data.split("@")[0] ?? "").slice(0, 200);
    return db.upsertUser({ email: email.data, name: Name.safeParse(local).data ?? "user"
});
};

/** The cookie a signed-in browser carries (E8.1). HttpOnly and SameSite=Strict: see
app.ts, where it is set. */
export const SESSION_COOKIE = "noon_session";

/**
    * E8.1 (F23): the caller is whoever holds a live sign-in session. The cookie's token is
    hashed and looked up;
    * a malformed one is never sent to the database. One indexed READ per request, no write
    (the E1.4 finding):
    * the expiry is fixed at sign-in, never slid forward.
    */
export const sessionIdentity: Identify = async (c, db) => {
    const token = getCookie(c, SESSION_COOKIE);
    return isSessionToken(token) ? db.userForSession(hashToken(token)) : undefined;
};

/** Development: a session first, so sign-in works there as it does everywhere, and the
header when there is none. */
const developmentIdentity: Identify = async (c, db) => (await sessionIdentity(c, db)) ??
devHeaderIdentity(c, db);

/**
    * Only the literal "development" turns the header on. "test" does not: an image built in
    CI and
    * promoted with NODE_ENV=test must not accept it. Tests pass the strategy they want
    directly.
    */
export const chooseIdentity = (nodeEnv: "development" | "test" | "production"): Identify
=>
    nodeEnv === "development" ? developmentIdentity : sessionIdentity;

```

`type Identify = (c, db) =>
Promise<User | undefined>`

`interface
AuthenticationProvider + a
bean`

a `Depends` callable

the strategy is a `REQUIRED` argument
of `buildApp`

profile-specific beans

dependency
overrides

A one-method interface in Java is just a function type here. And it is a *required* parameter with no default: forget it and the code does not compile, so development auth cannot reach production by omission.

FROM `apps/api/src/app.ts`

```
// Identity fails CLOSED: every route needs a caller unless it is listed here. A new
top-level
// route (E1.5's POST /documents/:id/session, for one) is protected without anyone
remembering to.
// createMiddleware carries the Variables type, so `c.var.user` is typed (not `any`)
downstream.
//
// E9.6 (F31): and every route is rate limited but the probes (an orchestrator refused
a probe kills a healthy
// process) and Gitea's webhook (the HMAC is its gate, and a refused delivery would
only be lost to the reconcile).
// A known caller is counted per user; a public route, or a caller nobody recognises,
per client address
// (client-address.ts: X-Forwarded-For only from a trusted proxy). Db.take is one
upsert, so every api instance
// shares the count. ponytail: one Postgres write per request (and a hot row per user,
held for one statement);
// ceiling: a few thousand requests a second on one primary; upgrade: Redis INCR,
failing open. With Postgres away the
// take throws and the answer is 500, closed: every one of these routes needs the
database anyway.
const UNLIMITED = new Set(["/health", "/ready", WEBHOOK_PATH]);
const PUBLIC_PATHS = new Set(["/health", "/ready", WEBHOOK_PATH, "/auth/signup",
"/auth/signin", "/auth/signout", "/auth/me"]);
// An IPv6 client is counted by its /64 (rateKey), or one host could rotate through
2^64 fresh counts.
const addressOf = (c: Context) => rateKey(clientAddress(peerOf(c), c.req.header("x-
forwarded-for"), trustProxy) ?? "unknown");
const byAddress = (c: Context) => db.take(`address:${addressOf(c)}`, limits.address);
const requireUser = createMiddleware<{ Variables: { user: User } }>(async (c, next) =>
{
  if (UNLIMITED.has(c.req.path)) return next();
  if (PUBLIC_PATHS.has(c.req.path)) {
    const verdict = await byAddress(c);
    return verdict.ok ? next() : limited(c, verdict.retryAfterSeconds);
  }
  const user = await identify(c, db);
  const verdict = await (user ? db.take(`user:${user.id}`, limits.user) :
```

```

byAddress(c));
  if (!verdict.ok) return limited(c, verdict.retryAfterSeconds);
  if (!user) return fail(c, 401, "unauthenticated");
  c.set("user", user);
  await next();
});
app.use("*", requireUser);

```

FROM `apps/api/src/app.ts`

```

// EVERYTHING about one org lives behind this middleware, including reading the org
itself: it is
// the one place that decides whether the caller may see this org. Not a member and no
such org
// are the same answer, 404, so a response never confirms that someone else's org
exists (F2).
// The same row carries the caller's role, which each route's `need` checks (E8.2).
const org = new Hono<{ Variables: { user: User; org: Org; role: Role; scope:
ReturnType<Db["forOrg"]> } }>();
org.use(async (c, next) => {
  const found = await db.getOrgForMember(c.req.param("orgId") ?? "", c.var.user.id);
  if (!found) return notFound(c);
  c.set("org", found.org);
  c.set("role", found.role);
  c.set("scope", db.forOrg(found.org.id));
  await next();
});

// With the caller's role: the members page shows an owner's controls by it, not by
finding them in the pages it has read.

```

404, never 403. A 403 says "this org exists, and you may not see it". That confirms a fact the caller had no right to. So "not a member" and "no such org" are *one query* and *one answer*. The test walks every per-org route as an outsider and compares status, bytes and headers with the answer for an org that does not exist.

CASE STUDY · FOUND BY REVIEW

Three ways the first version could have leaked, all found by review.

- **"Anything except production" is two values too many.** `NODE_ENV=test` also enabled the header. Now only the literal `"development"` does, and *unset means production*.
- **Auth was opt-in per path.** Only `/orgs/*` was guarded, so the next top-level route would have been public by default. Now every route needs a caller except an explicit allowlist. The very next bead added such a route and needed no auth code at all.
- **No test proved the real wiring.** Tests handed the strategy to `startServer` themselves, so `main.ts` could wire the wrong one and the whole gate stayed green. A new test runs the *real entry point in a child process* with a clean environment. It was mutation-checked: wire it wrong, the test goes red.

7 · A session token, and a function that returns its failure

`POST /documents/:id/session` answers "where do I connect for live editing, and with what?". The token is a standard HS256 JWT made with `node:crypto`. Two decisions differ from what a typical JWT library does by default.

FROM `packages/session-token/src/index.ts`

```
type VerifyResult =
  | { ok: true; claims: SessionClaims }
  | { ok: false; reason: "malformed" | "bad_signature" | "expired" | "wrong_document" };
```

That is a **discriminated union** (Lesson 0, section 3) used as a return type. A bad token is an *expected input*, not an exceptional event, so `verify` does not throw: it returns `{ ok: false, reason }`, and the compiler forces the caller to look at `ok` before it can touch `claims`.

TYPESCRIPT (THIS REPO)	JAVA / SPRING	PYTHON / FASTAPI
return a union: <code>{ok:true,...} {ok:false,reason}</code>	checked exception, or <code>Optional / Either</code>	raise, or return a tuple
the compiler forces the check	the compiler forces the catch (checked only)	nothing forces anything

FROM `packages/session-token/src/index.ts`

```
// Signature first, over the exact bytes received, in constant time: nothing in the
token is
// trusted (or even parsed) until this passes. The header must be OUR header, byte for
byte.
```

```
const given = Buffer.from(signature, "base64url");
// Node decodes base64url leniently (padding, "+" and "/", stray characters, the unused
bits of
// the last character), so several strings decode to one signature. Only the canonical
one may
// pass, or anything later keyed on the token string (a replay cache, a revocation
list) breaks.
const canonical = given.toString("base64url") === signature;
const matches = candidates.some((wanted) => given.length === wanted.length &&
timingSafeEqual(given, wanted));
if (header !== HEADER || !canonical || !matches) return { ok: false, reason:
"bad_signature" };
```

- **The algorithm is pinned.** The token's own `alg` header is never used to decide how to verify; the header must equal ours byte for byte. The classic attacks (`alg: none` , swapping the algorithm) have nothing to grab.
- **Constant-time comparison** (`timingSafeEqual`) before anything inside the token is parsed.
- **A list of secrets**, so a secret can be rotated without an outage: give verifiers `[new, old]` , switch the signer, wait out the 60-second lifetime, drop the old one.

CASE STUDY · FOUND BY REVIEW

Several different strings were the same token.

Node decodes base64url leniently: padding, `+` and `/` , stray characters and the unused bits of the last character all decode to the same bytes. A reviewer showed four equivalent last characters for one real token. Harmless today; but the day something is keyed on the token *string* (a replay cache, a revocation list), flipping one character walks around it. The `canonical` line above requires the one true encoding.

What is deliberately NOT in the token: the role. A token is checked once, when the socket opens, and the socket then lives for hours. A role inside it would be stale for the whole session. The sync server will read the role itself and be told when it changes (epic 8).

8 · Configuration, startup and shutdown

Environment variables are untrusted strings like any other input, so they get a schema too. There are **no defaults for secrets or addresses**: a missing value stops the process instead of pointing it at something with a well-known password.

FROM `apps/api/src/config.ts`

```
const Env = z.object({
  DATABASE_URL: DatabaseUrl,
  REDIS_URL: RedisUrl,
```

```

// Signs the tokens that open a document's WebSocket; the sync server holds the same
value.
SESSION_TOKEN_SECRET: z.string({ error: "SESSION_TOKEN_SECRET is required" }).min(32,
"SESSION_TOKEN_SECRET must be at least 32 characters"),
// The address BROWSERS use to reach the sync server (not the address inside the Docker
network). One URL for
// one node; with several (E7.1) the routing table `id=url,id=url`, ids as the nodes'
SYNC_NODE_ID.
SYNC_PUBLIC_URL: syncNodesField("SYNC_PUBLIC_URL"),
// Set when the app is reached through one public URL (a tunnel, noon-196): the
canvas's origin, whose dev
// server carries each preview as /preview/... . Unset (or empty): the canvas frames
the loopback address.
PREVIEW_PUBLIC_URL: z
  .string()
  .optional()
  .transform((value) => (value === "" ? undefined : value))
  .refine((value) => {
    if (value === undefined) return true;
    if (!URL.canParse(value)) return false;
    const url = new URL(value);
    // An ORIGIN: the preview's own path is appended to it, and nothing may ride along.
    return ["http:", "https:"].includes(url.protocol) && url.pathname === "/" &&
url.search === "" && url.hash === "" && url.username === "" && url.password === "";
  }, "PREVIEW_PUBLIC_URL must be an http(s) origin, like https://noon.example.com")
  .transform((value) => (value === undefined ? undefined : new URL(value).origin)),
// E5.3a: the secret Gitea signs push deliveries with (init.sh writes it to .env and
into the hook). Unset: the
// webhook answers 404 to everyone, and the git peer's reconcile alone notices pushes.
GITEA_WEBHOOK_SECRET: z
  .string()
  .optional()
  .transform((value) => (value === "" ? undefined : value))
  .refine((value) => value === undefined || value.length >= 32, "GITEA_WEBHOOK_SECRET
must be at least 32 characters"),
// Unset means production: the safe side. Anything that relaxes security must be asked
for by name.
NODE_ENV: z.enum(["development", "test", "production"]).default("production"),
PORT: port(3000),
// F31: AI runs an org may start per hour, whatever instance it asks. Decimal digits;
unset or empty: 60.
AI_RUNS_PER_HOUR: z
  .string()
  .optional()
  .transform((value) => (value === undefined || value === "" ? "60" : value))
  .pipe(z.string().regex(/^\\d+$/, "AI_RUNS_PER_HOUR must be decimal
digits")).transform(Number).pipe(z.number().int().min(1).max(100_000)),
// E9.6: the peers whose X-Forwarded-For names the client (client-address.ts). Unset or
empty: loopback only.
TRUST_PROXY: z
  .string()
  .optional()
  .transform((value, ctx) => {
    try {
      return trustedProxies(value === undefined || value === "" ? "loopback" : value);

```

```

    } catch (err) {
      ctx.addIssue({ code: "custom", message: `TRUST_PROXY: ${err as Error}.message`
    });
    return z.NEVER;
  }
  },
});

```

FROM `packages/process/src/index.ts`

```

export function createShutdown({ steps, timeoutMs, exit }: {
  steps: (() => Promise<void>)[];
  timeoutMs: number;
  exit: (code: number) => void;
}): () => Promise<void> {
  let running: Promise<void> | undefined;
  return () =>
    (running ??= (async () => {
      let timer: NodeJS.Timeout | undefined;
      const deadline = new Promise<"late">((resolve) => {
        timer = setTimeout(() => { resolve("late"); }, timeoutMs);
      });
      const work = (async () => {
        for (const step of steps) await step();
        return "done" as const;
      })().catch(() => "failed" as const);
      const outcome = await Promise.race([work, deadline]);
      clearTimeout(timer);
      exit(outcome === "done" ? 0 : 1);
    })());
}

```

Small file, four TypeScript idioms: `??=` (assign only if currently null/undefined) makes shutdown run *once*, so a second signal cannot crash it; `Promise.race` puts a deadline on the work (Docker sends SIGKILL ten seconds after SIGTERM); `as const` keeps `"done"` a literal type instead of widening to `string`; and `exit` is a parameter, which is why this is testable without killing the test runner.

In `main.ts` the steps are written `() => server.close()`, not `server.close`. Passing a method by itself detaches it from its object and loses `this`; the lint rule `unbound-method` caught exactly that. (An object of closures, section 2, has no such problem.)

PROBE	SPRING	FASTAPI
<code>/health</code> : is the process up? (never touches the database)	Actuator liveness	a liveness route
<code>/ready</code> : can it do its job? (<code>select 1</code>) drives the container healthcheck	Actuator readiness	a readiness route

CASE STUDY · FOUND BY REVIEW

Postgres was listening on every network interface, with its password in the repo.

Compose published the port on `0.0.0.0` and the superuser password was a committed default. On any shared network, anyone could have connected as the database owner. Now ports bind to `127.0.0.1`, compose contains no passwords and refuses to start without them, and `init.sh` generates random ones into the git-ignored `.env`.

9 · Least privilege in the database

The API does not connect as the database owner. A one-shot migrate step runs as the owner, applies migrations, and provisions a login role that can read and write data and do *nothing else*: no schema changes, no migration history, no temp tables, no other databases. Tests prove each refusal (`42501 insufficient_privilege`).

FROM `packages/db/src/index.ts`

```
function scramVerifier(password: string): string {
  const salt = randomBytes(16);
  const iterations = 4096;
  const salted = pbkdf2Sync(password, salt, iterations, 32, "sha256");
  const clientKey = createHmac("sha256", salted).update("Client Key").digest();
  const storedKey = createHash("sha256").update(clientKey).digest("base64");
  const serverKey = createHmac("sha256", salted).update("Server Key").digest("base64");
  return `SCRAM-SHA-
256${String(iterations)}:${salt.toString("base64")}${storedKey}:${serverKey}`;
}
```

Role DDL cannot take bind parameters, so a password would have to be written into the SQL text, where a failing statement copies it into the server log. Instead the client computes what Postgres would store anyway, a SCRAM verifier, and sends that. About ten lines of `node:crypto`.

CASE STUDY · FOUND BY REVIEW

Setting what you name is not the same as resetting.

The first provisioning ran `alter role ... nosuperuser nocreatedb`. A reviewer created a role with `BYPASSRLS`, replication and a dangerous role membership, ran our provisioning over it, and everything it had not named *survived*. The test could not have caught it: it always provisioned a brand-new role. Now every attribute is reset and memberships are revoked, and the test starts from a deliberately over-privileged role. *Test against the dirty state, not the clean one.*

10 · What the type system could not see

Every bug below passed `tsc` in strict mode, the linter and the first round of tests. They were found by a second pass whose only job was to refute "done". Types prove shapes; they say nothing about whitespace, time, memory, networks or privileges.

WHAT WENT WRONG	WHY	FIX
whitespace-only name stored, then unreadable forever	validation only on the read side	parse before the write; DB check on the trimmed value
NUL byte in a name became a 500	contract looser than Postgres	refuse control characters in <code>Name</code>
idle connection killed by Postgres crashed the process	no <code>pool.on('error')</code> listener	add one; log the message only (the object carries the password)
two processes migrating at once	bookkeeping table created outside the lock	create it inside the advisory lock
<code>-c search_path=\${schema}</code> took extra settings	libpq options are space-separated	plain identifiers only
200 MB body inflated memory 14x	no body limit	64 KB cap, 413
database open to the local network	port on 0.0.0.0 + committed password	127.0.0.1, generated secrets
role kept privileges across re-provisioning	set, not reset	reset everything; test a dirty role
<code>NODE_ENV=test</code> enabled dev auth	"not production" is too broad	one literal value; unset means production
next route would have been public	auth opt-in per prefix	fail closed with an allowlist
wrong wiring in main.ts stayed green	tests injected the dependency	run the real entry point in a child process
four strings verified as one token	lenient base64url decoding	require the canonical encoding

The habit to take away: after every test passes, ask "what production change would make this test fail?" and try it. If nothing does, the test proves nothing. That is mutation-checking, and it caught a test of mine that broke two rules at once and so could not notice when one was deleted.

Your turn: three drills

Do these alone. They live in `drills/lesson-1/` and both coded drills start RED; `sh drills/lesson-1/check.sh` shows your progress.

D1 • Extend a type, end to end. Give a workspace an optional `description` (at most 500 characters, no control characters, trimmed). You will touch the contract, a *new* migration, the db layer, and possibly the route. `make check` must stay green: the gate applies to you too.

Predict first: which of the four layers needs *no* change, and why? (Section 5 has the answer.)

D2 • Fix a planted bug. `drills/lesson-1/d2-cursor/cursor.ts` is a simplified copy of the paging code with one bug. Its test walks five rows created in the same millisecond and expects each exactly once. Form a theory before you read section 4's case study again.

D3 • Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ **1. Why does `createDb` return an object of closures instead of a class with a private field?**

TypeScript's `private` is erased at runtime, so it is a note to the compiler, not a wall. A closure variable is unreachable by anything: no reflection, no subclass, no `this` to lose when a method is passed as a callback. The tenancy rule is too important for a convention.

▼ **2. Why is a non-member answered with 404 rather than 403?**

403 confirms the org exists. "Not a member" and "no such org" are resolved by one query and return byte-identical responses, so the API never tells a caller anything about a tenant they do not belong to.

▼ **3. The session token names user, org and document. Why not the role too?**

A token is checked once, when the socket opens; the socket then lives for hours. A role baked into it would stay whatever it was at connect time, and epic 8 requires a role

change to reach open sessions within ten seconds. The sync server must read the role itself and be pushed changes.

Recall first

Look away. Trace `POST /orgs/:orgId/workspaces` through its five steps, and name the three layers that enforce tenancy. Then check.

▼ Check yourself

Identity middleware (401) → org middleware: member? (404) → body parsed against the contract (400 naming the field) → `db.forOrg(orgId)`, the only door → Postgres constraints. Tenancy is enforced by the closure (no unscoped query can be written), a compile-time `@ts-expect-error` test (a new door fails the build), and the composite foreign key (the database refuses).

Then test it

1. `export const Org = z.object({...})` and `export type Org = z.infer<typeof Org>` in one file. Why is that not a name clash?

The second declaration overrides the first

Values and types live in separate namespaces

Zod registers the name specially

It is a clash; the linter is configured to allow it

`Org.parse(x)` uses the value; `const o: Org` uses the type. The compiler always knows which position it is in.

2. The contract trims a name then requires $\text{length} \geq 1$. The database only requires $\text{length} \geq 1$. A client sends three spaces. What happened in the first version?

Rejected with a 400

Rejected by the database

Stored as an empty string

Stored, then every list for that org threw

Validation ran only on the read side. The row was storable but unreadable. Parse inputs with the same contract before writing.

3. What does `// @ts-expect-error` above `void db.pool;` prove?

The line is skipped by the compiler

Nothing; it silences an error

The pool throws at runtime if touched

The build fails if `db.pool` ever type-checks

Unlike `@ts-ignore`, it is an assertion: no error on the next line is itself an error (TS2578). A compile-time test.

4. Why does the paging cursor carry the timestamp as Postgres text instead of going through a JavaScript `Date` ?

Dates cannot be base64 encoded

Time zones differ between Node and Postgres

Text compares faster than timestamps

Date keeps milliseconds; timestamptz keeps microseconds

Rows created in the same millisecond would be skipped or repeated. That is Drill 2.

5. A handler writes `const { name } = await body(c, CreateOrgBody)` . Where does `name` 's type come from?

Inferred from the schema through the generic `S`

A cast inside `body()`

It is any

An annotation on the handler

`body<S extends z.ZodType>(c, schema: S): Promise<z.infer<S>>` : the return type is computed from the argument.

6. Auth was first applied to `/orgs/*` only. What was wrong with that?

Nothing; it was only slower

The next top-level route would be public by default

It ran the identity check twice

Wildcards do not match nested paths

Security defaults must fail closed: everything needs a caller except an explicit allowlist.

7. Tests passed `noIdentity` to `startServer` and saw 401. Why did that NOT prove production is safe?

The test ran with `NODE_ENV=test`

It never exercised what `main.ts` really wires

It did prove it

401 is the wrong status for production

An injected dependency proves the dependency, not the wiring. The fix runs the real entry point in a child process with a clean environment.

8. `verifySessionToken` returns `{ok:false, reason}` instead of throwing. Why?

Exceptions are slow in Node

So the error can be logged centrally

A bad token is expected input, and the union forces callers to check

TypeScript has no checked exceptions, so throwing is banned

TypeScript cannot say what a function throws. A discriminated union puts the failure in the type, where the compiler can see it.

Next: Lesson 2, after epic 2. One room per document, ops in one order, optimistic edits that reconcile, and a simulator that fuzzes the lot.